

Operációs rendszerek II.

Az operációs rendszerek biztonsági kérdései



Várkonyiné Kóczy Annamária

Professzor

Informatika Tanszék

[\(koczya@ujs.sk\)](mailto:koczya@ujs.sk)

varkonyi-koczy@uni-obuda.hu

Felhasznált irodalom:

- Kóczy-Kondorosi (szerk.): Operációs rendszerek mérnöki megközelítésben
- Tanenbaum: Modern Operating Systems 2nd. Ed.
- Silberschatz, Galvin, Gagne: Operating System Concepts



Tartalom

- A védelem célja
- A fenyegetés forrásai
- Belső biztonság
- Külső biztonság

A védelem célja

- Az operációs rendszer hardver és szoftver elemekből áll
- Minden elemnek egyéni azonosítója van és előre definiált műveletekkel érhető el
- *A védelmi probléma:* minden elemet csak az arra jogosult eljárások érhessenek el és azt helyesen használják

A fenyegetés forrásai

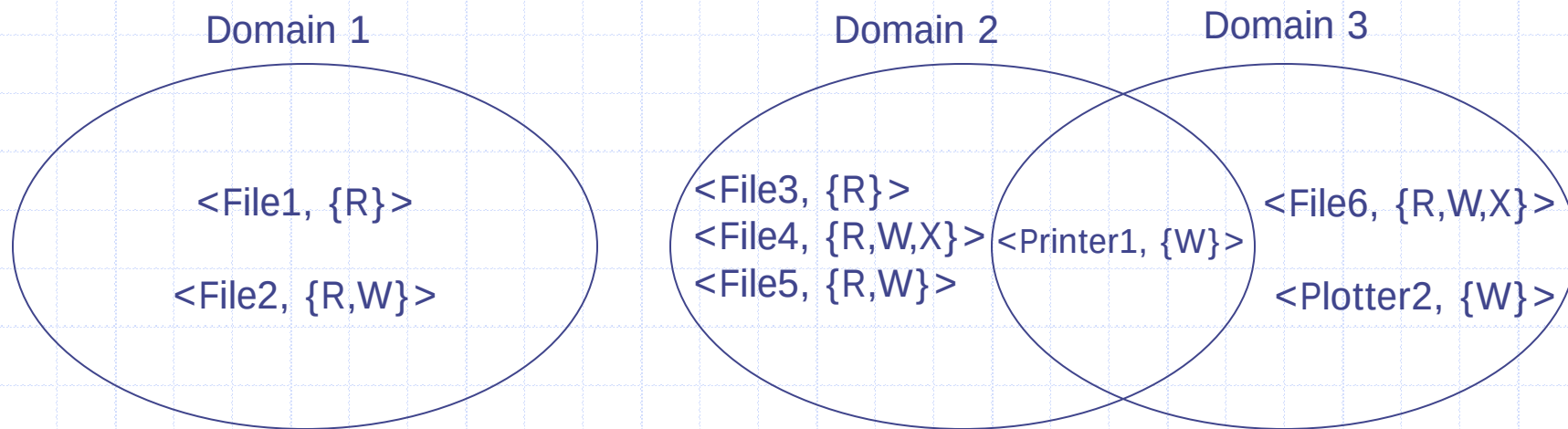
- Belső:
 - rendszerben lévő folyamatok felől,
 - Külső:
 - rendszertől független tényezőktől,
- (• HW hiba)

Belső biztonság

- Belső biztonsági rendszer (internal security):
(hogy adott módon használják az erőforrásokat)
 - programhibák elleni védelem,
 - erőforrások jogosulatlan felhasználásának védelme.
- Védelmi tartomány (protection domain): megadja, hogy a folyamat az adott állapotában mely erőforrásokhoz férhet hozzá és ezeken milyen műveleteket végezhet.
(melyik foly., melyik ef.-hoz, milyen módon férhet hozzá)
 - Minél szűkebbre kell definiálni, csak a valóban szükséges jogokat megadni (*need-to-know* elv).
 - Védelmi tartomány lehet statikus vagy dinamikus (állapottal változó).

Védelmi tartományok

- Hozzáférési jogok:
<Objektum, {jogosultságok listája}>,
ahol a jogosultságok listája az adott objektumon végrehajtható műveletek egy részhalmaza
- Védelmi tartomány (domain): hozzáférési jogok halmaza



Pl: Egy domain 2-ben futó folyamat a File5 objektumon Read és Write műveleteket hajthat végre.

Statikus védelmi tartomány

- A "jogok" a folyamat élete során nem változnak.
- Hozzáférési mátrix:
 - sorai a tartományok,
 - oszlopai az erőforrások (ritka mátrix).

		Object							
		File1	File2	File3	File4	File5	File6	Printer1	Plotter2
Domain	1	Read	Read Write						
	2			Read	Read Write Execute	Read Write		Write	
	3						Read Write Execute	Write	Write

Statikus védelmi tartomány ábrázolása

- Globális tábla (global table)
- Hozzáférési lista (access control list)
- Jogosítványok listája (capability list)

Globális tábla

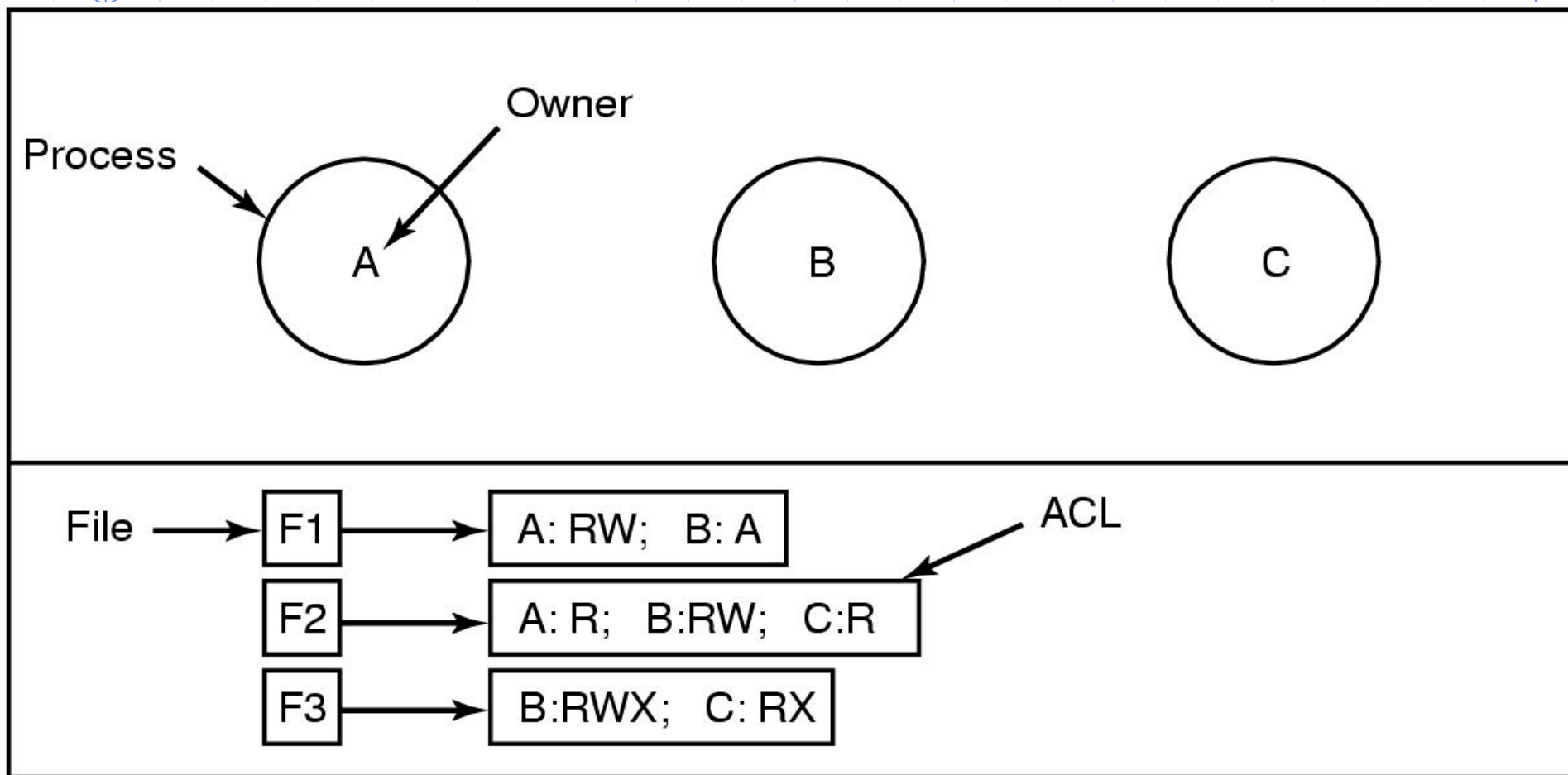
- (global table)
- A <tartomány, erőforrás, művelet> hármassokot tartalmazza.
- Minden művelet esetén a táblát végig kell nézni.
- A tábla túl nagy, így a keresés lassú.
- Nem lehet csoportokat képezni (pl. mindenki számára elérhető erőforrásokot minden tartományhoz fel kell venni).

Hozzáférési lista (access control list)

- Minden **erőforráshoz** <tartomány, művelet> párok sorozata + alapértelmezett műveletek (minden tartomány számára engedélyezett művelet).
- Felhasználói igényekhez igazodnak, új erőforrás létrehozásakor lehet definiálni a védelmi tartományokban kiadható műveleteket.

		Object							
		File1	File2	File3	File4	File5	File6	Printer1	Plotter2
Domain	1	Read	Read Write						
	2			Read	Read Write Execute	Read Write		Write	
	3						Read Write Execute	Write	Write

Hozzáférési lista 2.

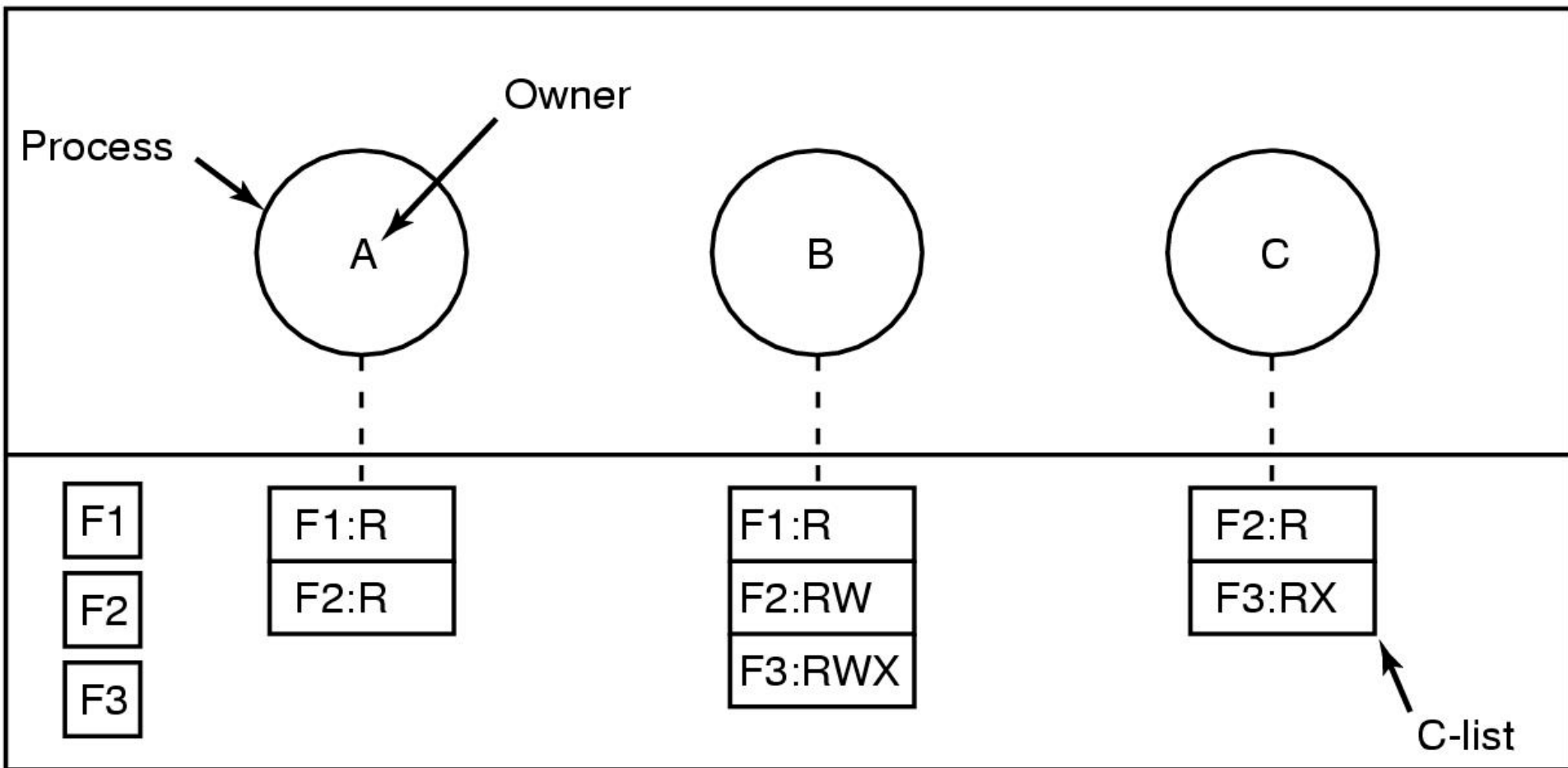


Jogosítványok listája (capability list)

- Minden **tartomány**hoz <erőforrás, művelet> párok sorozata. Jogosítvány egy <erőforrás, művelet> párra vonatkozó referencia. A tartományhoz tartozó folyamatok számára közvetlenül nem érhetők el.
- Nem az objektumokhoz igazodik, nem szemléletes.
- Az erőforrás használatakor a folyamat csak bemutatja a jogosítványát, az OS-nek csak ellenőrizni kell az érvényességét (nincs szükség keresésekre).

		Object							
		File1	File2	File3	File4	File5	File6	Printer1	Plotter2
Domain	1	Read	Read Write						
	2			Read	Read Write Execute	Read Write		Write	
	3						Read Write Execute	Write	Write

Jogosítványok listája



Zár-kulcs (lock-key) módszer

- zár/erőforrás (bitsorozat)
- kulcs(ok)/védelmi tartomány (bitsorozat)
- Ha a kulcs „nyitja” (kiegészíti) a zárat, akkor a tartományt birtokló folyamat elvégezheti a kulcshoz tartozó műveletet
- Átmenetnek tekinthető az objektumelérési lista és a tartományok jogosítványainak listája módszerek között

Statikus védelmi tartományok megvalósítása

- Gyakori a hozzáférési és jogosítvány listák együttes használata.
- Pl. UNIX rendszerekben egy állomány megnyitásánál a folyamatnak meg kell adnia a műveleteket. Ezt a hozzáférési lista alapján ellenőrzi az OS, majd a folyamat kap egy jogosítványt, a későbbi műveleteknél ezt használja.

Dinamikus védelmi tartomány

- A statikus védelmi tartományokban túl sok erőforrás van (mindet fel kell venni), a dinamikus védelmi tartomány csak a szükségeseket tartalmazza.
- Hozzáférési mátrix itt is használható.
- A védelmi tartomány is egy erőforrás, ezt is fel kell venni a mátrix oszlopai közé.

Hozzáférési mátrix dinamikus védelmi tartományokban

Védelmi tartomány, mint erőforrás

Művelet

	Object									
	File1	File2	File3	File4	File5	File6	Printer1	Plotter2	Domain1	Domain2
domain	1	Read	Read Write							Switch
	2		Read	Read Write Execute	Read Write		Write			
	3					Read Write Execute	Write	Write		

Pl.: Egy Domain1-ben futó folyamat átkapcsolhat Domain2-re.

Dinamikus védelmi tartomány

Új bejegyzések hozzáférési mátrixban:

- kapcsolás (switch)
 - Az adott tartományból egy folyamat átkapcsolhat az oszlopban kijelölt tartományra.
- másolás (copy)
 - Az adott tartományban tartózkodó folyamat hozzáférési jogait átmásolhatja egy másik tartományba. A másoláson kívül lehet: *átadás* (transfer), amikor a jogok az eredeti tartományból törlődnek; *korlátozott másolás*, az új tartományból a jogot nem lehet továbbadni.
- tulajdonos (owner)
 - Az erőforrás tulajdonosa más tartományokban törölhet vagy adhat az erőforráshoz tartozó műveleteket (oszlopok mentén történő módosítások).
- vezérlő (control)
 - Sorok mentén történő módosításokra ad lehetőséget. Csak a tartományokhoz tartozó oszlopokban szerepelhet. Ha egy tartomány vezérli a másikat, akkor annak sorában elhelyezhet illetve törölhet jogosultságokat.

Dinamikus védelmi tartomány megvalósítása

- MULTICS

- File 0 (XY, *, RWX)

user csoport hozzáférési
id. id. jogok

- File 4 (Z, *, ---)

kizárhatunk felhasználókat vagy csoportokat

- UNIX

- type rwx r-x r-x

file: - user csoport ált.

dir: d jog. jog. jog.

Külső biztonság

az idegen behatolók ellen

- A felhasználók azonosítása
- Naplózás (veszélyeztetett pontok figyelése)
- Rejtjelezés (titkosítás, digitális aláírás)

A felhasználók azonosítása

- Jogtalan hozzáférések megakadályozására a felhasználókat egyértelműen és megbízhatóan azonosítani kell:
 - a felhasználó attribútumai (ujjlenyomat, retina, stb.)
 - a felhasználó birtokában lévő tárgyak (kulcs, azonosító kártya, stb.)
 - felhasználó által tudott ismeretek (pl. név, jelszó)
- Jelenlegi rendszerek főleg jelszóval azonosítanak.
 - biztonságos jelszó (jó jelszó választás, jelszó gyakori cseréje)
 - rendszer szigorúan védett állományban tárolja (rejtjelezett)

A jelszó kezelése

- A rendszer a jelszót biztonsági okokból nem tárolja (ellopható)
- A p jelszó helyett $f(p)$ -t tároljuk, ahol f „egyirányú” függvény
- $f(p)$ sok esetben nyilvános
- Támadási módok:
 - Gyakori jelszavakkal való próbálkozás (pl. szótár). Nagy valószínűséggel néhány jelszót talál a jelszófájlban.
 - Figyelem: ez nem egy adott felhasználó ellen irányul; cél bárkinek a jelszavát megszerezni.
- Módosítás: „sózás” (jelszó+véletlen karakter sorozat titkosítva)

Sózott jelszó

- A jelszófájl nem $f(p)$ -t tárolja (hiszen p esetleg gyakori szó!)
- Tárolunk
 - egy véletlen N egész számot, ez a „só”, és
 - $f(pN)$ -t.
- Pl. (jelszó: alma)
User2, 67832, $f(\text{alma}67832)$,
ahol
 - User2 a felhasználó azonosítója,
 - 67832 a só,
 - $f(\text{alma}67832)$ az eltárolt kódolt sózott jelszó.
- Nehezebb a „bárki ellen” való véletlen próbálkozás

Naplózás

- A rendszer feljegyzi
 - az egyes erőforrásokhoz a hozzáférések időpontját (+ hány próbálkozás),
 - a műveletet és
 - a felhasználót.
- Ez nem véd, de az elkövető utólagos azonosítását megkönnyíti.

Rejtjelzés

- A nem védett információs csatornán átadott adatok rejtjelezéssel (encryption) védhetjük.
- Két kulcsos rejtjelezés (csapóajtó kódok). Ide tartozik még az üzenethitelesítés (az üzenetet senki nem módosította)
- partner hitelesítés (a küldő hitelesítése), elektronikus aláírás.

A rosszindulatú programok és jellemzőik

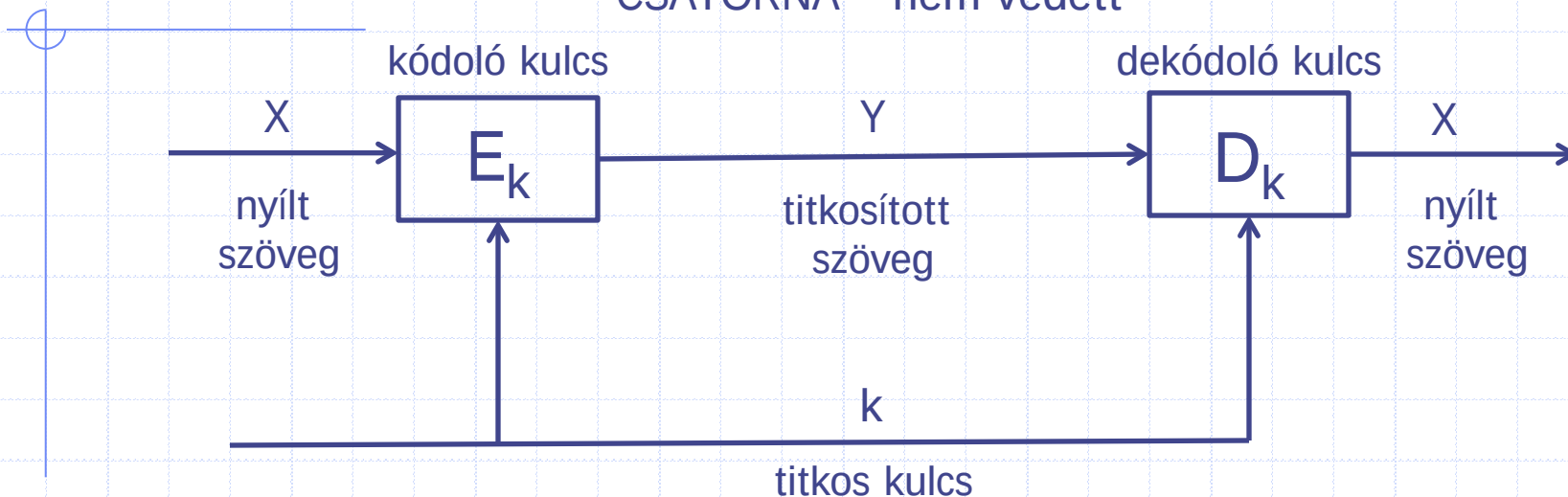
- Trójai faló (Trojan horse)
 - A keresési utakat felhasználva indulnak el, a rendszer a kívánt program előtt találja meg azonos néven a falóvat, vagy
 - gyakran elgépelt parancsok használata (pl. ls → la)
- Rejtekajtó (trap door / back door)
 - Egy programban az írója a programban olyan részeket tesz amelyek nem dokumentált, jogosulatlan tevékenységeket végezhetnek.
 - Léteznek fordítóprogramokba, vagy programkönyvtárakba épített rejtekajtók, amelyek a lefordított kódba is belekerülnek.
- Féreg (Worm)
 - Önmagát indítja el több példányban, így leterheli a rendszer központi erőforrásait.

A rosszindulatú programok és jellemzőik

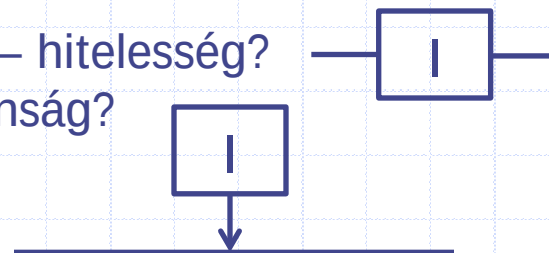
- Vírus
 - Más programokba saját utasításait csempészi. A program elindul, ezek is futnak: terjedés, kártétel
- Stack és Buffer Overflow támadás
 - A programban jelen levő, nem ellenőrzött korlátok kihasználásával a vezérlés saját kódnak átadása (pl. hosszú inputba csempészett kód).
- Denial of Service támadás
 - A célzott gép túlterhelése, ezáltal a hasznos tevékenység megakadályozása

Nyilvános kulcsú titkosítás

CSATORNA – nem védett



Támadó: - aktív (kicseréli a szöveget) – hitelesség?
- passzív (hallgatózik) – biztonság?



E_k - nyilvános

D_k - titkos

E_k, D_k – "olcsó" k -ból

D_k - E_k -ből „drága”

egyirányú számítás

Nyilvános kulcsú titkosítás – Téglapélda

- Szupernövekvő sorozat: $l_i > \sum_{j=1}^{i-1} l_j$ pl. 1,2,5,9, ...
 - A kódolt szöveg az összhossz: $Y' = E_k'(X)$, $X = D_k'(Y')$
 - könnyű kódolni, pl. mikor, hová megyünk?

— kedden — moziba
— szombaton — korcsolyázni

- Kedden korcsolyázni megyünk:

- de dekódolni is: lépésenként megkeresem a leghosszabb téglát, ami belefér

— ... — marad — Ebbe melyik fér még bele?

Ezért a szupernövekvő sorozat helyett:

Nyilvános kulcsú titkosítás – Téglapélda (folyt.)

- A szupernövekvő sorozat legyen a „k” kulcs (Y')
 - Keresünk w , m számokat, hogy $(w, m) = 1$
 - $Y = w * Y' \bmod m$, (Y már nem szupernövekvő!)
 - Dekódolás: $w^{-1} * Y \bmod m = Y'$; $X = D'_k(Y')$
- Nyilvános: $Y_i - k$
 - $Y_i = w * Y'_i \bmod m$; $(w, m) = 1$;
 - w „összekeveri” a téglahosszokat az $(1, m-1)$ tartományban
 - Ha $(w, m) = 1$, bizonyítható, hogy $\exists w^{-1}$
 - w, m nagy számok, csak nagyjából van bizonyítva, hogy relatív prímek

Nyilvános kulcsú titkosítás – Téglapélda (folyt.)

- hosszú téglák, $n=100$ féle üzenet
 $l_i' \in [(2^{i-1} - 1)2^{100} + 1; 2^{i-1}2^{100}]$
- $l_1' \in [1; 2^{100}]$
- $l_2' \in [2^{100} + 1; 2 \cdot 2^{100}]$
- $l_{100}' \in [2^{201} + 1; 2^{202} - 1]$
- $l_i := w \cdot l_i' \bmod m$
- Eredeti hossz: 100 bit (1- benne van, 0 – nincs)
- Kódolt hossz: $100 * 202 \rightarrow 209$ bit

↓
 2^7

100 bitből 209 bitet csinál

Digitális aláírás (hitelesítés)

- $X = D_k\{E_k\{X\}\}$

- általában E_k és D_k sorrendje nem fordítható meg
- de ha igen, nagy jelentőségű

- Digitális aláírás (2 cél: tőle jött + ne tudja letagadni)

A $\rightarrow$ B

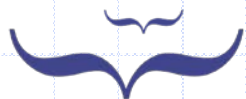
- A: kódolás - $E_B(A, D_A(X))$

- csak B tudja dekódolni $\rightarrow$ A nem szedheti le az aláírását ($D_A(X)$)

- B: dekódolás

- $D_B(E_B(A, D_A(X))) = I(A, D_A(X)) \rightarrow D_A(X) \rightarrow$ A-tól jött
- $E_A(D_A(X)) = X$

- $X = E_A(D_B(E_B(D_A(X))))$



Hosszú üzenetek küldése

- Blokkokban küldés, tetszőleges útvonalon, érkezési sorrend?
- Komplet blokkok kicserélhetők korábban megfigyelt és lemásolt blokkokkal
- A hitelesítés egy X' kontrol szumma blokk hozzátételével és küldésével történik
- Eredeti üzenet: $X_{\text{part1}}, X_{\text{part2}}, X_{\text{part3}}, \dots, X_{\text{partn}}$
- Elküldött üzenet: $X_{\text{part1}}, X_{\text{part2}}, X_{\text{part3}}, \dots, X_{\text{partn}}, X'$
- $X' = f(X, \text{aláírás}, \text{dátum})$
- Vevő: bevár minden blokkot, sorrendbe rakja, dekódolja, hitelesíti X' segítségével
- Ha a támadó kicserél valamit, X' nem fog stimmelni